

Python Pandas Cheat Sheet

A practical quick reference for beginners & intermediate users
Ordered from the most commonly used functions to the least

EVERY EXAMPLE IN THIS GUIDE USES THIS DATAFRAME

```
import pandas as pd

df = pd.read_csv("HR_ListOfEmployees.csv")
```

THE 14 COLUMNS IN THE DATASET

Employee_ID	Full_Name	Gender	Department
Position	Hired_Date	Employment_Type	Office_Location
Salary	Age	Email	Phone_Number
Performance_Rating	Manager		

PREVIEW — df.head()

Employee_ID	Full_Name	Gender	Department	Position	Hired_Date
PH-1466	Alvin Aglipay	Male	Engineering	Data Engineer	4-Dec-2022
PH-1207	Alvin Alcantara	Male	Engineering	Network Engineer	31-Jan-2022
PH-1674	Alvin Luna	Male	Engineering	Data Engineer	5-Mar-2026
PH-1492	Alvin Ramos	Male	Engineering	Data Engineer	30-Mar-2022
PH-1429	Amalia Alonzo	Female	Engineering	Data Engineer	23-Apr-2024

Blue bar = code you type

Green bar = real output returned

150 examples · **14** topics

Every sample output on the following pages was produced by actually running the code against the dataset.

Table of Contents

01	Importing Data Load a dataset into pandas.	8 functions	3
02	Viewing Data Get a fast first look at your table.	12 functions	4
03	Selecting Rows & Columns Pick the exact data you need.	10 functions	6
04	Filtering Data Keep only the rows that match a rule.	12 functions	8
05	Sorting Data Order rows from highest to lowest.	8 functions	10
06	Adding & Removing Columns Create, rename and delete columns.	10 functions	11
07	Handling Missing Values Find and fix empty cells.	11 functions	13
08	Data Cleaning Fix duplicates, types and messy text.	13 functions	15
09	Grouping & Aggregation Summarise data by category.	16 functions	17
10	Merging & Joining Combine two tables into one.	8 functions	20
11	Reshaping Data Change the shape of your table.	8 functions	21
12	Working with Dates Handle hire dates and time periods.	12 functions	23
13	Exporting Data Save your results to a file.	8 functions	25
14	Useful Utilities Small helpers that save time.	14 functions	26

1. Importing Data

8 functions

Load a dataset into pandas.

1.1 `import pandas as pd` # Standard way to import pandas

SAMPLE OUTPUT

2.2.3

1.2 `df = pd.read_csv("HR_ListOfEmployees.csv")` # Reads a CSV file into a DataFrame

SAMPLE OUTPUT

(250, 14)

1.3 `pd.read_csv("HR_ListOfEmployees.csv", nrows=3)` # Reads only the first few rows

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department	Position
0	PH-1466	Alvin Aglipay	Engineering	Data Engineer
1	PH-1207	Alvin Alcantara	Engineering	Network Engineer
2	PH-1674	Alvin Luna	Engineering	Data Engineer

1.4 `pd.read_csv(f, usecols=["Full_Name", "Salary"])` # Loads selected columns only

SAMPLE OUTPUT

	Full_Name	Salary
0	Alvin Aglipay	69500.0
1	Alvin Alcantara	91500.0
2	Alvin Luna	100000.0
...	...	...
247	Wilfredo Magsaysay	86500.0
248	Yolanda Mendoza	51500.0
249	Zenaida Valdez	38000.0

1.5 `pd.read_csv(f, parse_dates=["Hired_Date"])` # Reads dates as real date values

SAMPLE OUTPUT

dtype('<M8[ns]')

1.6 `pd.read_excel("HR_ListOfEmployees.xlsx")` # Reads an Excel file

SAMPLE OUTPUT

(250, 14)

1.7 `pd.read_csv(f, index_col="Employee_ID")` # Uses a column as the row index

SAMPLE OUTPUT

Index(['PH-1466', 'PH-1207', 'PH-1674', 'PH-1492'], dtype='object', name='Employee_ID')

1.8 `pd.DataFrame({"Name": ["Ana"], "Age": [30]})` # Builds a DataFrame from a dictionary

SAMPLE OUTPUT

	Name	Age
0	Ana	30

2. Viewing Data

12 functions

Get a fast first look at your table.

2.1 `df.head()` # Shows the first 5 rows

SAMPLE OUTPUT

	Employee_ID	Full_Name	Gender	Department	Position	Hired_Date
0	PH-1466	Alvin Aglipay	Male	Engineering	Data Engineer	4-Dec-2022
1	PH-1207	Alvin Alcantara	Male	Engineering	Network Engineer	31-Jan-2022
2	PH-1674	Alvin Luna	Male	Engineering	Data Engineer	5-Mar-2026
3	PH-1492	Alvin Ramos	Male	Engineering	Data Engineer	30-Mar-2022
4	PH-1429	Amalia Alonzo	Female	Engineering	Data Engineer	23-Apr-2024

2.2 `df.tail(3)` # Shows the last 3 rows

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department	Position	Salary
247	PH-1376	Wilfredo Magsaysay	Sales	Sales Manager	86500.0
248	PH-1644	Yolanda Mendoza	Sales	Account Executive	51500.0
249	PH-1472	Zenaida Valdez	SALES	Account Executive	38000.0

2.3 `df.shape` # Shows rows and columns

SAMPLE OUTPUT

```
(250, 14)
```

2.4 `df.columns` # Lists all column names

SAMPLE OUTPUT

```
Index(['Employee_ID', 'Full_Name', 'Gender', 'Department', 'Position', 'Hired_Date',
      'Employment_Type', 'Office_Location', 'Salary', 'Age', 'Email', 'Phone_Number',
      'Performance_Rating', 'Manager'],
      dtype='object')
```

2.5 `df.info()` # Summary of columns, types, nulls

SAMPLE OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250 entries, 0 to 249
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Employee_ID           250 non-null   object
1   Full_Name             250 non-null   object
2   Gender                250 non-null   object
3   Department            250 non-null   object
4   Position              250 non-null   object
5   Hired_Date            250 non-null   object
6   Employment_Type       250 non-null   object
7   Office_Location       250 non-null   object
8   Salary                238 non-null   float64
9   Age                  250 non-null   int64
10  Email                 245 non-null   object
11  Phone_Number          250 non-null   object
12  Performance_Rating    241 non-null   float64
13  Manager               244 non-null   object
dtypes: float64(2), int64(1), object(11)
memory usage: 27.5+ KB
```

2.6 `df.describe()` # Statistics for numeric columns

SAMPLE OUTPUT

	Salary	Age	Performance_Rating
count	238.0	250.0	241.0
mean	82766.8	40.8	3.5
std	26409.5	10.4	0.8
...	...	...	...
50%	80250.0	41.0	3.5
75%	96375.0	48.8	4.1
max	184000.0	59.0	5.0

2.7 `df.dtypes` # Shows the data type per column

SAMPLE OUTPUT

```
Employee_ID      object
Full_Name        object
Gender           object
Department       object
...
Email            object
Phone_Number     object
Performance_Rating float64
Manager          object
```

2.8 `len(df)` # Counts the number of rows

SAMPLE OUTPUT

```
250
```

2.9 `df.sample(3)` # Picks random rows

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department	Salary
60	PH-1157	Amalia Bautista	FINANCE	110000.0
20	PH-1535	Grace Dela Rosa	Engineering	115500.0
202	PH-1861	Josefina Guzman	Operations	38500.0

2.10 `df.describe(include="object")` # Statistics for text columns

SAMPLE OUTPUT

	Gender	Department	Employment_Type
count	250	250	250
unique	2	13	3
top	Female	Engineering	Full-Time
freq	129	57	181

2.11 `df.index` # Shows the row index range

SAMPLE OUTPUT

```
RangeIndex(start=0, stop=250, step=1)
```

2.12 `df.size` # Total number of cells

SAMPLE OUTPUT

```
3500
```

3. Selecting Rows & Columns

10 functions

Pick the exact data you need.

3.1 `df["Salary"]` # Selects one column

SAMPLE OUTPUT

```
0      69500.0
1      91500.0
2     100000.0
3     102000.0
...
246    111500.0
247     86500.0
248     51500.0
249     38000.0
```

3.2 `df[["Full_Name", "Department", "Salary"]]` # Selects several columns

SAMPLE OUTPUT

	Full_Name	Department	Salary
0	Alvin Aglipay	Engineering	69500.0
1	Alvin Alcantara	Engineering	91500.0
2	Alvin Luna	Engineering	100000.0
...	...	...	...
247	Wilfredo Magsaysay	Sales	86500.0
248	Yolanda Mendoza	Sales	51500.0
249	Zenaida Valdez	SALES	38000.0

3.3 `df.loc[0]` # Selects a row by label

SAMPLE OUTPUT

Employee_ID	PH-1466
Full_Name	Alvin Aglipay
Gender	Male
Department	Engineering
...	...
Email	alvin.aglipay@apex...
Phone_Number	+63 991 369 9524
Performance_Rating	3.6
Manager	Ramon Villanueva

3.4 `df.iloc[0]` # Selects a row by position

SAMPLE OUTPUT

Employee_ID	PH-1466
Full_Name	Alvin Aglipay
Gender	Male
Department	Engineering
...	...
Email	alvin.aglipay@apex...
Phone_Number	+63 991 369 9524
Performance_Rating	3.6
Manager	Ramon Villanueva

3.5 `df.loc[0:2, ["Full_Name", "Salary"]]` # Rows and columns by name

SAMPLE OUTPUT

	Full_Name	Salary
0	Alvin Aglipay	69500.0
1	Alvin Alcantara	91500.0
2	Alvin Luna	100000.0

3.6 `df.iloc[0:3, 0:4]` # Rows and columns by number

SAMPLE OUTPUT

	Employee_ID	Full_Name	Gender	Department
0	PH-1466	Alvin Aglipay	Male	Engineering
1	PH-1207	Alvin Alcantara	Male	Engineering
2	PH-1674	Alvin Luna	Male	Engineering

```
3.7 df.at[0, "Full_Name"] # Gets one single cell fast
```

SAMPLE OUTPUT

```
Alvin Aglipay
```

```
3.8 df.iloc[-1] # Selects the very last row
```

SAMPLE OUTPUT

```
Employee_ID      PH-1472
Full_Name        Zenaida Valdez
Gender           Female
Department       SALES
...
Email            zenaida.valdez@ape...
Phone_Number     +63 911 898 3096
Performance_Rating 3.9
Manager          Patricia Navarro
```

```
3.9 df["Salary"].head(4).tolist() # Converts a column to a list
```

SAMPLE OUTPUT

```
[69500.0, 91500.0, 100000.0, 102000.0]
```

```
3.10 df.iloc[[0, 5, 10]] # Selects specific rows by position
```

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department
0	PH-1466	Alvin Aglipay	Engineering
5	PH-1577	Amalia Fernandez	Engineering
10	PH-1720	Bernadette Guzman	ENGINEERING

4. Filtering Data

12 functions

Keep only the rows that match a rule.

4.1 `df[df["Department"] == "Engineering"]` # Filters rows by one value

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department	Position
0	PH-1466	Alvin Aglipay	Engineering	Data Engineer
1	PH-1207	Alvin Alcantara	Engineering	Network Engineer
2	PH-1674	Alvin Luna	Engineering	Data Engineer
3	PH-1492	Alvin Ramos	Engineering	Data Engineer

4.2 `df[df["Salary"] > 100000]` # Filters rows using a number

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department	Salary
3	PH-1492	Alvin Ramos	Engineering	102000.0
4	PH-1429	Amalia Alonzo	Engineering	106500.0
7	PH-1049	Andrea Reyes	Engineering	111000.0
10	PH-1720	Bernadette Guzman	ENGINEERING	112500.0

4.3 `df[(df["Salary"] > 80000) & (df["Age"] < 30)]` # Two conditions, both must be true

SAMPLE OUTPUT

	Full_Name	Department	Salary	Age
4	Amalia Alonzo	Engineering	106500.0	24
19	Fernando Ilagan	Engineering	93500.0	22
22	Hector Zamora	ENGINEERING	91000.0	25
32	Lourdes Fernandez	Engineering	81000.0	24

4.4 `df[(df["Age"] < 25) | (df["Age"] > 58)]` # Two conditions, either can be true

SAMPLE OUTPUT

	Full_Name	Age	Department
4	Amalia Alonzo	24	Engineering
5	Amalia Fernandez	59	Engineering
11	Bernadette Mendoza	22	Engineering
19	Fernando Ilagan	22	Engineering

4.5 `df[df["Office_Location"].isin(["Cebu City", "Clark"])]` # Keeps rows matching a list

SAMPLE OUTPUT

	Full_Name	Office_Location	Salary
0	Alvin Aglipay	Clark	69500.0
8	Bea Soriano	Clark	81000.0
15	Dennis Mendoza	Cebu City	128500.0
16	Divina Salvador	Clark	104500.0

4.6 `df[df["Age"].between(30, 35)]` # Filters values inside a range

SAMPLE OUTPUT

	Full_Name	Age	Position
21	Hector Luna	31	Data Engineer
24	Ignacio Ocampo	30	QA Engineer
25	Jasmine Cabrera	30	QA Engineer
40	Oscar Zamora	31	Software Engineer

4.7 `df[df["Position"].str.contains("Manager")]` # Filters text that contains a word

SAMPLE OUTPUT

	Full_Name	Position	Salary
15	Dennis Mendoza	Engineering Manager	128500.0
17	Divina Soriano	Engineering Manager	150000.0
27	Kevin Valdez	Engineering Manager	118500.0
46	Rico Villanueva	Engineering Manager	91000.0

4.8 `df[~(df["Employment_Type"] == "Full-Time")]` # Excludes rows using NOT

SAMPLE OUTPUT

	Full_Name	Employment_Type	Department
0	Alvin Aglipay	Part-Time	Engineering
4	Amalia Alonzo	Contractor	Engineering
6	Amalia Ocampo	Contractor	Engineering
8	Bea Soriano	Contractor	Engineering

4.9 `df.query("Salary > 150000 and Age < 40")` # Filters using a written condition

SAMPLE OUTPUT

	Full_Name	Salary	Age
53	Teresita Espiritu	159000.0	33
56	Wilfredo Mendoza	184000.0	27

4.10 `df[df["Salary"].isnull()]` # Finds rows with missing values

SAMPLE OUTPUT

	Employee_ID	Full_Name	Salary
13	PH-1643	Cristina Alcantara	NaN
55	PH-1697	Vicente Zamora	NaN
70	PH-1480	Erlinda Cabrera	NaN
74	PH-1623	Kristine Cruz	NaN

4.11 `df[df["Full_Name"].str.startswith("Alvin")]` # Filters text by its start

SAMPLE OUTPUT

	Full_Name	Department	Salary
0	Alvin Aglipay	Engineering	69500.0
1	Alvin Alcantara	Engineering	91500.0
2	Alvin Luna	Engineering	100000.0
3	Alvin Ramos	Engineering	102000.0

4.12 `df.loc[df["Gender"] == "Female", "Salary"].mean()` # Filters then computes a value

SAMPLE OUTPUT

82352.46

5. Sorting Data

8 functions

Order rows from highest to lowest.

5.1 `df.sort_values("Salary")` # Sorts by a column, smallest first

SAMPLE OUTPUT

	Full_Name	Department	Salary
188	Bea Cruz	Operations	33000.0
103	Divina Espiritu	Human Resources	34000.0
190	Bea Quijano	Operations	34000.0
207	Nestor Santos	Operations	37000.0

5.2 `df.sort_values("Salary", ascending=False)` # Sorts from highest to lowest

SAMPLE OUTPUT

	Full_Name	Department	Salary
56	Wilfredo Mendoza	Engineering	184000.0
135	Fernando Castillo	IT	160000.0
53	Teresita Espiritu	Engineering	159000.0
17	Divina Soriano	Engineering	150000.0

5.3 `df.sort_values(["Department", "Salary"])` # Sorts by two columns

SAMPLE OUTPUT

	Full_Name	Department	Salary
22	Hector Zamora	ENGINEERING	91000.0
10	Bernadette Guzman	ENGINEERING	112500.0
44	Ramon Navarro	Engineering	61500.0
5	Amalia Fernandez	Engineering	62500.0

5.4 `df.nlargest(3, "Salary")` # Gets the top 3 rows

SAMPLE OUTPUT

	Full_Name	Position	Salary
56	Wilfredo Mendoza	Engineering Manager	184000.0
135	Fernando Castillo	IT Manager	160000.0
53	Teresita Espiritu	Engineering Manager	159000.0

5.5 `df.nsmallest(3, "Age")` # Gets the bottom 3 rows

SAMPLE OUTPUT

	Full_Name	Age	Department
11	Bernadette Mendoza	22	Engineering
19	Fernando Ilagan	22	Engineering
152	Patricia Ocampo	22	IT

5.6 `df.sort_values("Full_Name").head(3)` # Sorts text alphabetically

SAMPLE OUTPUT

	Full_Name	Department
0	Alvin Aglipay	Engineering
1	Alvin Alcantara	Engineering
59	Alvin Bautista	Finance

5.7 `df.sort_index(ascending=False)` # Sorts rows by the index

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department
249	PH-1472	Zenaida Valdez	SALES
248	PH-1644	Yolanda Mendoza	Sales
247	PH-1376	Wilfredo Magsaysay	Sales
246	PH-1722	Vicente Uy	Sales

5.8 `df["Salary"].rank(ascending=False).head(4)` # Ranks values by position

SAMPLE OUTPUT

0	161.5
1	70.5
2	56.0
3	53.0

6. Adding & Removing Columns

10 functions

Create, rename and delete columns.

6.1 `df["Monthly_Pay"] = df["Salary"] / 12` # Creates a new calculated column

SAMPLE OUTPUT

	Full_Name	Salary	Monthly_Pay
0	Alvin Aglipay	69500.0	5791.666667
1	Alvin Alcantara	91500.0	7625.000000
2	Alvin Luna	100000.0	8333.333333

6.2 `df["Country"] = "Philippines"` # Adds a column with one value

SAMPLE OUTPUT

	Full_Name	Country
0	Alvin Aglipay	Philippines
1	Alvin Alcantara	Philippines
2	Alvin Luna	Philippines

6.3 `df.drop(columns=["Phone_Number", "Email"])` # Removes columns

SAMPLE OUTPUT

```
Index(['Employee_ID', 'Full_Name', 'Gender', 'Department', 'Position', 'Hired_Date'],
      dtype='object')
```

6.4 `df.drop(index=[0, 1])` # Removes rows by index

SAMPLE OUTPUT

	Employee_ID	Full_Name	Gender	...	Phone_Number	Performance_Rating	Manager
2	PH-1674	Alvin Luna	Male	...	+63 911 942 2255	3.0	Ramon Villanueva
3	PH-1492	Alvin Ramos	Male	...	+63 977 426 2394	3.0	Ramon Villanueva

6.5 `df.rename(columns={"Full_Name": "Name"})` # Renames a column

SAMPLE OUTPUT

```
Index(['Employee_ID', 'Name', 'Gender', 'Department'], dtype='object')
```

6.6 `df["Senior"] = df["Age"] > 50` # Creates a True/False column

SAMPLE OUTPUT

	Full_Name	Age	Senior
0	Alvin Aglipay	40	False
1	Alvin Alcantara	37	False
2	Alvin Luna	36	False

6.7 `df["Level"] = np.where(df["Salary"] > 90000, "High", "Low")` # Adds a column using a condition

SAMPLE OUTPUT

	Full_Name	Salary	Level
0	Alvin Aglipay	69500.0	Low
1	Alvin Alcantara	91500.0	High
2	Alvin Luna	100000.0	High

6.8 `df.assign(Bonus=df["Salary"] * 0.10)` # Adds a column without changing df

SAMPLE OUTPUT

	Full_Name	Salary	Bonus
0	Alvin Aglipay	69500.0	6950.0
1	Alvin Alcantara	91500.0	9150.0
2	Alvin Luna	100000.0	10000.0

6.9 `df.insert(1, "Row_No", range(1, len(df) + 1))` # Inserts a column at a position

SAMPLE OUTPUT

	Row_No	Employee_ID	Full_Name
0	1	PH-1466	Alvin Aglipay
1	2	PH-1207	Alvin Alcantara
2	3	PH-1674	Alvin Luna

6.10 `df["Initials"] = df["Full_Name"].str[0] # Builds a column from text`

SAMPLE OUTPUT

	Full_Name	Initials
0	Alvin Aglipay	A
1	Alvin Alcantara	A
2	Alvin Luna	A

7. Handling Missing Values

11 functions

Find and fix empty cells.

7.1 `df.isnull().sum()` # Counts missing values per column

SAMPLE OUTPUT

```
Employee_ID      0
Full_Name        0
Gender           0
Department       0
..
Email            5
Phone_Number     0
Performance_Rating 9
Manager         6
```

7.2 `df.isnull().sum().sum()` # Counts all missing values

SAMPLE OUTPUT

```
32
```

7.3 `df["Salary"].isnull().sum()` # Counts blanks in one column

SAMPLE OUTPUT

```
12
```

7.4 `df.dropna()` # Removes rows with any blank

SAMPLE OUTPUT

```
(218, 14)
```

7.5 `df.dropna(subset=["Salary"])` # Removes rows missing one column

SAMPLE OUTPUT

```
(238, 14)
```

7.6 `df["Salary"].fillna(0)` # Replaces blanks with a value

SAMPLE OUTPUT

```
0      69500.0
1      91500.0
2     100000.0
3     102000.0
```

7.7 `df["Salary"].fillna(df["Salary"].mean())` # Fills blanks with the average

SAMPLE OUTPUT

```
0
```

7.8 `df["Manager"].fillna("Not Assigned")` # Fills blank text with a label

SAMPLE OUTPUT

```
Manager
Ramon Villanueva    57
Nestor Fernandez    39
Teresita Ocampo     38
```

7.9 `df["Salary"].ffill()` # Copies the value from above

SAMPLE OUTPUT

```
0      69500.0
1      91500.0
2     100000.0
3     102000.0
```

7.10 `df.dropna(axis=1)` # Removes columns that have blanks

SAMPLE OUTPUT

```
Index(['Employee_ID', 'Full_Name', 'Gender', 'Department', 'Position', 'Hired_Date',  
      'Employment_Type', 'Office_Location', 'Age', 'Phone_Number'],  
      dtype='object')
```

7.11 `df["Email"].notnull().sum()` # Counts rows that are filled

SAMPLE OUTPUT

```
245
```

8. Data Cleaning

13 functions

Fix duplicates, types and messy text.

8.1 `df.duplicated().sum()` # Counts duplicate rows

SAMPLE OUTPUT

```
2
```

8.2 `df.drop_duplicates()` # Removes duplicate rows

SAMPLE OUTPUT

```
(248, 14)
```

8.3 `df.drop_duplicates(subset=["Employee_ID"])` # Removes repeats by one column

SAMPLE OUTPUT

```
(247, 14)
```

8.4 `df["Office_Location"].str.strip()` # Removes extra spaces from text

SAMPLE OUTPUT

```
['Clark', 'Manila', 'Davao City', 'Iloilo City']
```

8.5 `df["Department"].str.title().replace({"It": "IT"})` # Fixes inconsistent letter casing

SAMPLE OUTPUT

```
['Engineering', 'Finance', 'Human Resources', 'IT', 'Marketing', 'Operations', 'Sales']
```

8.6 `df["Email"].str.lower()` # Converts text to lowercase

SAMPLE OUTPUT

```
0    alvin.aglipay@apex...
1    alvin.alcantara@ap...
2    alvin.luna@apexsol...
```

8.7 `df["Phone_Number"].str.replace(" ", "")` # Replaces text inside values

SAMPLE OUTPUT

```
0    +639913699524
1    +639588654885
2    +639119422255
```

8.8 `df["Salary"].astype("Int64")` # Changes a column data type

SAMPLE OUTPUT

```
0    69500
1    91500
2   100000
3   102000
```

8.9 `df.columns.str.lower()` # Cleans all column names

SAMPLE OUTPUT

```
Index(['employee_id', 'full_name', 'gender', 'department', 'position', 'hired_date'],
      dtype='object')
```

8.10 `df["Salary"].round(-3)` # Rounds numbers for reporting

SAMPLE OUTPUT

```
0    70000.0
1    92000.0
2   100000.0
3   102000.0
```

8.11 `df["Gender"].replace({"Male": "M", "Female": "F"})` # Replaces values with new ones

SAMPLE OUTPUT

```
Gender
F    129
M    121
```

8.12 `df["Full_Name"].str.split(" ").str[0]` # Splits text into parts

SAMPLE OUTPUT

```
0    Alvin
1    Alvin
2    Alvin
```

8.13 `pd.to_numeric(df["Age"], errors="coerce")` # Converts text into numbers

SAMPLE OUTPUT

```
0    40
1    37
2    36
3    41
```


9. Grouping & Aggregation

16 functions

Summarise data by category.

9.1 `df["Department"].value_counts()` # Counts rows per category

SAMPLE OUTPUT

```
Department
Engineering    59
IT              40
Finance         38
Operations      32
Sales           31
Human Resources 27
Marketing       23
```

9.2 `df["Salary"].mean()` # Calculates the average

SAMPLE OUTPUT

```
82766.81
```

9.3 `df.groupby("Department")["Salary"].mean()` # Average per group

SAMPLE OUTPUT

```
Department
Engineering    98114.035088
Finance         88986.111111
Human Resources 76740.740741
IT              79657.894737
Marketing       73886.363636
Operations      67375.000000
Sales           76383.333333
```

9.4 `df.groupby("Department").size()` # Counts employees per group

SAMPLE OUTPUT

```
Department
Engineering    59
Finance         38
Human Resources 27
IT              40
Marketing       23
Operations      32
Sales           31
```

9.5 `df.groupby("Office_Location")["Salary"].sum()` # Total per group

SAMPLE OUTPUT

```
Office_Location
Cagayan de Oro  2117000.0
Cebu City       2966000.0
Clark           2179000.0
Davao City      2498000.0
Iloilo City     1504500.0
Manila          4943500.0
Quezon City     3490500.0
```

9.6 `df.groupby("Gender")["Salary"].agg(["min", "max", "mean"])` # Several statistics at once

SAMPLE OUTPUT

```

      min      max      mean
Gender
Female  33000.0  159000.0  82352.459016
Male    37000.0  184000.0  83202.586207
```

9.7 `df.groupby(["Department", "Gender"])["Salary"].mean()` # Groups by two columns

SAMPLE OUTPUT

Department	Gender	
Engineering	Female	99187.500000
	Male	96740.000000
Finance	Female	90083.333333
	Male	87888.888889
Human Resources	Female	78346.153846
	Male	75250.000000

9.8 `df["Salary"].sum()` # Adds up all values

SAMPLE OUTPUT

19698500.0

9.9 `df["Salary"].median()` # Finds the middle value

SAMPLE OUTPUT

80250.0

9.10 `df["Department"].nunique()` # Counts unique categories

SAMPLE OUTPUT

13

9.11 `df["Department"].unique()` # Shows unique values

SAMPLE OUTPUT

['Engineering', 'Finance', 'Human Resources', 'IT', 'Marketing', 'Operations', 'Sales']

9.12 `df.groupby("Employment_Type")["Salary"].count()` # Counts non-empty values per group

SAMPLE OUTPUT

Employment_Type	
Contractor	29
Full-Time	174
Part-Time	35

9.13 `df.groupby("Department")["Salary"].max()` # Highest value per group

SAMPLE OUTPUT

Department	
Engineering	184000.0
Finance	135500.0
Human Resources	132000.0
IT	160000.0
Marketing	129000.0
Operations	109500.0
Sales	148000.0

9.14 `df["Department"].value_counts(normalize=True)` # Shows percentage share

SAMPLE OUTPUT

Department	
Engineering	0.236
IT	0.160
Finance	0.152
Operations	0.128
Sales	0.124

9.15 `df.groupby("Department")["Salary"].transform("mean")` # Adds group average to each row

SAMPLE OUTPUT

0	98114.035088
1	98114.035088
2	98114.035088
3	98114.035088

```
9.16 # Named summary columns
df.groupby("Department").agg(Staff=("Employee_ID", "count"),
                             Avg=("Salary", "mean"))
```

SAMPLE OUTPUT

	Staff	Avg
Department		
Engineering	59	98114.0
Finance	38	88986.0
Human Resources	27	76741.0
IT	40	79658.0

10. Merging & Joining

8 functions

Combine two tables into one.

10.1 `pd.merge(df, bonus, on="Employee_ID")` # Joins tables on a shared key

SAMPLE OUTPUT

	Employee_ID	Full_Name	Bonus
0	PH-1466	Alvin Aglipay	5000
1	PH-1207	Alvin Alcantara	7000
2	PH-1674	Alvin Luna	6500

10.2 `pd.merge(df, bonus, on="Employee_ID", how="left")` # Keeps all rows of the left table

SAMPLE OUTPUT

	Employee_ID	Full_Name	Bonus
0	PH-1466	Alvin Aglipay	5000.0
1	PH-1207	Alvin Alcantara	7000.0
2	PH-1674	Alvin Luna	6500.0
3	PH-1492	Alvin Ramos	NaN

10.3 `pd.merge(df, dept_head, on="Department", how="left")` # Adds lookup info to each row

SAMPLE OUTPUT

	Full_Name	Department	Head
0	Alvin Aglipay	Engineering	R. Villanueva
1	Alvin Alcantara	Engineering	R. Villanueva
2	Alvin Luna	Engineering	R. Villanueva

10.4 `pd.concat([df.head(2), df.tail(2)])` # Stacks tables on top of each other

SAMPLE OUTPUT

	Employee_ID	Full_Name	Department
0	PH-1466	Alvin Aglipay	Engineering
1	PH-1207	Alvin Alcantara	Engineering
248	PH-1644	Yolanda Mendoza	Sales
249	PH-1472	Zenaida Valdez	SALES

10.5 `pd.concat([a, b], axis=1)` # Joins tables side by side

SAMPLE OUTPUT

	Employee_ID	Full_Name	Salary	Age
0	PH-1466	Alvin Aglipay	69500.0	40
1	PH-1207	Alvin Alcantara	91500.0	37
2	PH-1674	Alvin Luna	100000.0	36

10.6 `df.merge(bonus, on="Employee_ID", how="outer")` # Keeps rows from both tables

SAMPLE OUTPUT

	Employee_ID	Full_Name	Bonus
248	PH-1989	Jasmine Jacinto	NaN
249	PH-1994	Andrea Domingo	NaN
250	PH-9999	NaN	8000.0

10.7 `df.merge(bonus, on="Employee_ID", indicator=True)` # Shows where each row came from

SAMPLE OUTPUT

	Employee_ID	Bonus	_merge
0	PH-1466	5000	both
1	PH-1207	7000	both

10.8 `a.join(b)` # Joins two tables by index

SAMPLE OUTPUT

	Full_Name	Salary
0	Alvin Aglipay	69500.0
1	Alvin Alcantara	91500.0
2	Alvin Luna	100000.0

11. Reshaping Data

8 functions

Change the shape of your table.

```
11.1 # Builds an Excel-style pivot table
pd.pivot_table(df, values="Salary", index="Department",
               columns="Gender", aggfunc="mean")
```

SAMPLE OUTPUT

Gender	Female	Male
Department		
Engineering	99188.0	96740.0
Finance	90083.0	87889.0
Human Resources	78346.0	75250.0
IT	73395.0	85921.0
Marketing	74958.0	72600.0

```
11.2 pd.crosstab(df["Department"], df["Employment_Type"]) # Counts rows in a grid
```

SAMPLE OUTPUT

Employment_Type	Contractor	Full-Time	Part-Time
Department			
Engineering	9	43	7
Finance	6	26	6
Human Resources	1	24	2
IT	3	32	5
Marketing	5	14	4

```
11.3 # Averages by one category
df.pivot_table(index="Office_Location", values="Salary",
               aggfunc="mean")
```

SAMPLE OUTPUT

Office_Location	Salary
Cagayan de Oro	92043.0
Cebu City	84743.0
Clark	77821.0
...	...
Iloilo City	79184.0
Manila	85233.0
Quezon City	77567.0

```
11.4 small.melt(id_vars="Full_Name", var_name="Field") # Turns columns into rows
```

SAMPLE OUTPUT

	Full_Name	Field	value
0	Alvin Aglipay	Salary	69500.0
1	Alvin Alcantara	Salary	91500.0
2	Alvin Aglipay	Age	40.0
3	Alvin Alcantara	Age	37.0

```
11.5 df.head(3).T # Flips rows and columns
```

SAMPLE OUTPUT

	0	1	2
Employee_ID	PH-1466	PH-1207	PH-1674
Full_Name	Alvin Aglipay	Alvin Alcantara	Alvin Luna
Gender	Male	Male	Male
Department	Engineering	Engineering	Engineering
Position	Data Engineer	Network Engineer	Data Engineer

```
11.6 grouped.unstack() # Moves an index level to columns
```

SAMPLE OUTPUT

Gender	Female	Male
Department		
Engineering	99188.0	96740.0
Finance	90083.0	87889.0
Human Resources	78346.0	75250.0
IT	73395.0	85921.0

```
11.7 df.set_index("Employee_ID").head(3) # Uses a column as the index
```

SAMPLE OUTPUT

Employee_ID	Full_Name	Salary
PH-1466	Alvin Aglipay	69500.0
PH-1207	Alvin Alcantara	91500.0
PH-1674	Alvin Luna	100000.0

```
11.8 df.reset_index(drop=True).head(3) # Rebuilds a clean number index
```

SAMPLE OUTPUT

	Employee_ID	Full_Name
0	PH-1466	Alvin Aglipay
1	PH-1207	Alvin Alcantara
2	PH-1674	Alvin Luna

12. Working with Dates

12 functions

Handle hire dates and time periods.

12.1 `df["Hired_Date"] = pd.to_datetime(df["Hired_Date"])` # Converts text into real dates

SAMPLE OUTPUT

```
0    2022-12-04
1    2022-01-31
2    2026-03-05
```

12.2 `df["Hired_Date"].dt.year` # Extracts the year

SAMPLE OUTPUT

```
0    2022
1    2022
2    2026
3    2022
```

12.3 `df["Hired_Date"].dt.month_name()` # Extracts the month name

SAMPLE OUTPUT

```
0    December
1    January
2     March
3     March
```

12.4 `df["Hired_Date"].dt.day_name()` # Extracts the weekday name

SAMPLE OUTPUT

```
0    Sunday
1    Monday
2    Thursday
3    Wednesday
```

12.5 `df[df["Hired_Date"].dt.year == 2022]` # Filters rows by year

SAMPLE OUTPUT

```
Full_Name Hired_Date
0    Alvin Aglipay 2022-12-04
1    Alvin Alcantara 2022-01-31
3     Alvin Ramos 2022-03-30
```

12.6 # Calculates years of service
`df["Years"] = (pd.Timestamp("today") - df["Hired_Date"]).dt.days // 365`

SAMPLE OUTPUT

```
Full_Name Hired_Date Years
0    Alvin Aglipay 2022-12-04    3
1    Alvin Alcantara 2022-01-31    4
2     Alvin Luna 2026-03-05    0
```

12.7 `df["Hired_Date"].dt.strftime("%b-%Y")` # Formats a date as text

SAMPLE OUTPUT

```
0    Dec-2022
1    Jan-2022
2    Mar-2026
3    Mar-2022
```

12.8 `df["Hired_Date"].dt.to_period("M").value_counts().head(3)` # Counts hires per month

SAMPLE OUTPUT

```
Hired_Date
2015-05    7
2016-03    6
2023-01    6
Freq: M
```

```
12.9 df["Hired_Date"].min(), df["Hired_Date"].max()    # Finds earliest and latest dates
```

SAMPLE OUTPUT

```
(Timestamp('2015-01-05 00:00:00'), Timestamp('2026-03-27 00:00:00'))
```

```
12.10 df[df["Hired_Date"] > "2024-01-01"]    # Filters dates after a date
```

SAMPLE OUTPUT

```
      Full_Name Hired_Date
2    Alvin Luna 2026-03-05
4  Amalia Alonzo 2024-04-23
7   Andrea Reyes 2024-01-09
```

```
12.11 df.set_index("Hired_Date").resample("YE").size()    # Counts records per year
```

SAMPLE OUTPUT

```
Hired_Date
2015-12-31    21
2016-12-31    24
2017-12-31    16
2018-12-31    28
Freq: YE-DEC
```

```
12.12 df["Hired_Date"].dt.quarter    # Gets the quarter number
```

SAMPLE OUTPUT

```
0    4
1    1
2    1
3    1
```


13. Exporting Data

8 functions

Save your results to a file.

13.1 `df.to_csv("clean_employees.csv", index=False)` # Saves the table as CSV

SAMPLE OUTPUT

```
(250, 14)
```

13.2 `df.to_excel("employees.xlsx", index=False)` # Saves the table as Excel

SAMPLE OUTPUT

```
(250, 14)
```

13.3 `summary.to_csv("summary.csv")` # Saves a summary table

SAMPLE OUTPUT

```

      Department  Salary
0      Engineering 98114.0
1           Finance 88986.0
2  Human Resources 76741.0
```

13.4 `df.head(2).to_json(orient="records")` # Converts the table to JSON

SAMPLE OUTPUT

```
[{"Employee_ID": "PH-1466", "Full_Name": "Alvin Aglipay", "Salary": 69500.0}, {"Employee_ID": "PH-1207", "Full_Name": "Alvin Alcantara", "Salary": 91500.0}]
```

13.5 `df.head(2).to_dict(orient="records")` # Converts rows into dictionaries

SAMPLE OUTPUT

```
[{'Employee_ID': 'PH-1466', 'Full_Name': 'Alvin Aglipay'}, {'Employee_ID': 'PH-1207', 'Full_Name': 'Alvin Alcantara'}]
```

13.6 `df.head(3).to_markdown(index=False)` # Creates a Markdown table

SAMPLE OUTPUT

```

| Employee_ID | Full_Name | Salary |
|:-----:|:-----:|:-----:|
| PH-1466 | Alvin Aglipay | 69500 |
| PH-1207 | Alvin Alcantara | 91500 |
| PH-1674 | Alvin Luna | 100000 |
```

13.7 `df.to_excel(writer, sheet_name="HR", index=False)` # Writes to a named Excel sheet

SAMPLE OUTPUT

```
['HR']
```

13.8 `df.to_csv("pipe.csv", sep="|", index=False)` # Saves with a custom separator

SAMPLE OUTPUT

```
Employee_ID|Full_Name|Gender|Department|Position|Hired_Date|
```

14. Useful Utilities

14 functions

Small helpers that save time.

14.1 `df.copy()` # Makes a safe copy of data

SAMPLE OUTPUT

```
(250, 14)
```

14.2 `pd.set_option("display.max_columns", None)` # Shows every column when printing

SAMPLE OUTPUT

```
(no value returned - the file is written to disk)
```

14.3 `df["Salary"].apply(lambda x: x * 1.1)` # Applies a function to a column

SAMPLE OUTPUT

```
0    76450.0
1   100650.0
2   110000.0
3   112200.0
```

14.4 # Maps values to new labels
`df["Employment_Type"].map({"Full-Time": "FT",
 "Part-Time": "PT",
 "Contractor": "CTR"})`

SAMPLE OUTPUT

```
0    PT
1    FT
2    FT
3    FT
```

14.5 `df.nunique()` # Unique values in every column

SAMPLE OUTPUT

```
Employee_ID      247
Full_Name        247
Gender           2
Department       13
Position         29
Hired_Date       237
Employment_Type   3
```

14.6 `df[["Salary", "Age", "Performance_Rating"]].corr()` # Checks how columns relate

SAMPLE OUTPUT

	Salary	Age	Performance_Rating
Salary	1.00	-0.04	-0.04
Age	-0.04	1.00	-0.08
Performance_Rating	-0.04	-0.08	1.00

14.7 `df["Salary"].describe()` # Quick stats for one column

SAMPLE OUTPUT

```
count      238.0
mean      82767.0
std       26410.0
min       33000.0
25%       65500.0
50%       80250.0
75%       96375.0
max      184000.0
```

14.8 `df.memory_usage(deep=True).sum()` # Shows memory used in bytes

SAMPLE OUTPUT

```
176563
```

```
14.9 df.sample(frac=0.02, random_state=1) # Takes a random percentage sample
```

SAMPLE OUTPUT

	Employee_ID	Full_Name
67	PH-1311	Carmela Espiritu
249	PH-1472	Zenaida Valdez
230	PH-1639	Gabriel Mendoza
161	PH-1844	Yolanda Dagohoy
91	PH-1724	Teresita Uy

```
14.10 df.equals(df.copy()) # Checks if two tables match
```

SAMPLE OUTPUT

```
True
```

```
14.11 df["Salary"].clip(lower=40000, upper=150000) # Limits values to a range
```

SAMPLE OUTPUT

```
0    69500.0
1    91500.0
2   100000.0
3   102000.0
```

```
14.12 df.select_dtypes(include="number").columns # Lists only numeric columns
```

SAMPLE OUTPUT

```
Index(['Salary', 'Age', 'Performance_Rating'], dtype='object')
```

```
14.13 df.style.format({"Salary": "{:,.0f}"}) # Formats numbers for display
```

SAMPLE OUTPUT

```
Styler object -> renders in Jupyter / Excel
```

```
14.14 pd.options.display.float_format = "{:,.2f}".format # Sets a global number format
```

SAMPLE OUTPUT

```
58,432.75
```